Step 2 — The Graph: Answer or Escalate — Solution

> Back to the step · Hint 1 · Hint 2

Story: this builds the whole behavior as an explicit LangGraph structure — retrieve, route on the confidence gate, then answer or escalate as separate named nodes. **If not:** the answer-or-escalate decision would stay buried inside a plain `if` statement, invisible and un-pausable, instead of being the visible, inspectable graph structure it genuinely needs to be.

The state

```
# src/state.py
from typing import TypedDict

class SupportState(TypedDict):
    question: str
    context: str
    best_distance: float
    answer: str
    escalated: bool

def start_state(question):
    return {
        "question": question,
        "context": "",
        "best_distance": 2.0,
        "answer": "",
        "escalated": False,
    }
```

The graph

```
# src/graph.py
# retrieve -> route on the confidence gate -> answer (grounded) OR escalate
# (interrupt, pausing for a human). The route is the answer-or-escalate choice.
from langchain_core.prompts import ChatPromptTemplate
```

```
from langgraph.graph import END, START, StateGraph
from langgraph.types import interrupt

from src.confidence import is_confident
from src.rag.retriever import best_match
from src.state import SupportState

ANSWER_PROMPT = (
    "Answer the user's question using ONLY the help article below. Be brief "
    "and do not add anything not in it.\n\nHelp article:\n{context}\n\n"
    "Question: {question}"
)


def build_graph(model, collection, cutoff=0.8):
    def retrieve_node(state):
        text, distance = best_match(collection, state["question"])
        return {"context": text, "best_distance": distance}

    def route(state):
        if is_confident(state["best_distance"], cutoff):
            return "answer"
        return "escalate"

    def answer_node(state):
        prompt = ChatPromptTemplate.from_template(ANSWER_PROMPT)
        reply = (prompt | model).invoke({
            "context": state["context"], "question": state["question"],
        })
        return {"answer": reply.content, "escalated": False}

    def escalate_node(state):
        human = interrupt({"question": state["question"]})   # PAUSE here
        return {"answer": human, "escalated": True}

    graph = StateGraph(SupportState)
    graph.add_node("retrieve", retrieve_node)
    graph.add_node("answer", answer_node)
    graph.add_node("escalate", escalate_node)
    graph.add_edge(START, "retrieve")
    graph.add_conditional_edges("retrieve", route,
                                {"answer": "answer", "escalate": "escalate"})
    graph.add_edge("answer", END)
    graph.add_edge("escalate", END)
    return graph
```

How it works

- **`retrieve_node`** searches the help docs and writes `context` + `best_distance` into state. It never decides anything — it just gathers the evidence.
- **`route`** is the whole decision, and it's Step 1's gate: `is_confident(best_distance)` → `"answer"` or `"escalate"`. Because it's a conditional edge, the decision is *visible* in the graph, not buried in a function.
- **`answer_node`** answers using **only** the matched article — the gate already confirmed it's relevant, so grounding the reply in it is safe.
- **`escalate_node`** calls `interrupt({...})`, which **pauses the entire graph** and surfaces the question to whatever is driving it (Step 3). Its return value — the human's reply — becomes the answer once the graph resumes.

Note `build_graph` returns the *uncompiled* graph — Step 3 compiles it with a checkpointer, which is what `interrupt` needs to pause against.

Tests for this step: route a confident state (`best_distance = 0.2`) and confirm it returns `"answer"` ; route a non-confident one (`1.5`) and confirm `"escalate"` . The full live run (real model, real pause/resume) is Step 3.